

ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВОЙ ПРОЕКТ

Тема: «Проектирование и разработка модуля бронирования оборудования»

ПО ПМ.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП-23/1
[Фамилия Имя Отчество]

Руководитель:
Трушина Александра Игоревна

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....3
1 АНАЛИЗ ЗАДАЧИ БРОНИРОВАНИЯ ОБОРУДОВАНИЯ....5
1.1 Как используется общее оборудование....5
1.2 Обзор аналогов....5
1.3 Требования к модулю....6
1.4 Выбор средств разработки....7
Выводы по разделу 1....8
2 ПРОЕКТИРОВАНИЕ....9
2.1 Модель данных....9
2.2 Главное правило и граничные случаи....10
2.3 Состояния брони....11
2.4 Архитектура....13
2.5 Алгоритм создания брони....14
Выводы по разделу 2....14
3 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ....15
3.1 Структура проекта и каталог оборудования....15
3.2 Проверка доступности и создание брони....16
3.3 Отмена, текущие и завершённые брони....18
3.4 Хранение данных и развёртывание....19
3.5 Тестирование программного модуля....20
Выводы по разделу 3....23
ЗАКЛЮЧЕНИЕ....24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....25
ПРИЛОЖЕНИЕ А. ЛИСТИНГ ПРАВИЛ БРОНИРОВАНИЯ....27
ПРИЛОЖЕНИЕ Б. ЛИСТИНГ ТЕСТОВ ПРАВИЛ БРОНИРОВАНИЯ....28

ВВЕДЕНИЕ

В колледже и в любой организации есть общее оборудование: проекторы, ноутбуки, фотоаппараты, микрофоны. Им пользуются разные сотрудники, и каждый раз возникает вопрос, свободен ли проектор на третью пару и кто взял ноутбук на выездное мероприятие. Пока бронирование ведётся устно, в общем чате или в бумажном журнале у лаборанта, двойные брони и споры «кто первый» неизбежны.

Актуальность темы определяется тем, что дорогостоящее оборудование должно использоваться без простоев и без конфликтов, а ответственный за него сотрудник – видеть полную картину: что занято, кем, на какой срок и что сейчас на обслуживании.

Степень разработанности. Задачи бронирования ресурсов решаются календарями общего доступа (Outlook, «Яндекс Календарь») и специализированными системами учёта, однако календари не знают о состоянии оборудования и не мешают создать пересекающуюся бронь, а специализированные системы избыточны для одного колледжа.

Объект исследования – процесс использования общего оборудования сотрудниками образовательной организации.

Предмет исследования – программный модуль бронирования оборудования с проверкой доступности на выбранный период.

Цель работы – разработать на платформе .NET 10 веб-модуль, который ведёт каталог оборудования, позволяет выбрать период использования, создавать и отменять брони, проверяет доступность оборудования и показывает текущие и завершённые бронирования.

Задачи:

– изучить процесс бронирования и сформулировать требования, главное правило и ограничения;

– спроектировать модель данных, состояния брони, архитектуру и алгоритм создания брони;

– реализовать каталог, проверку доступности, создание и отмену броней, списки и график загрузки;

– проверить модуль автоматическими тестами и ручными проверками;

– подготовить инструкции для запуска и для пользователя.

Методы: анализ предметной области, моделирование данных и состояний, проектирование алгоритма, модульное тестирование, функциональное тестирование в браузере.

Теоретическая значимость – формализация условия пересечения периодов и граничных случаев бронирования. Практическая значимость – модуль готов к работе в колледже: запускается одной командой, данные хранятся в одном файле SQLite.

Работа состоит из введения, трёх разделов, заключения, списка источников и приложений.

1 АНАЛИЗ ЗАДАЧИ БРОНИРОВАНИЯ ОБОРУДОВАНИЯ

1.1 Как используется общее оборудование

Оборудование колледжа числится на балансе как основные средства и имеет инвентарный номер, присвоенный при постановке на учёт [2]. За него отвечает материально ответственное лицо – лаборант или сотрудник ИТ-отдела. Преподаватели и сотрудники берут оборудование на время: на пару, на мероприятие, на выездную работу приёмной комиссии.

Без единой системы процесс выглядит так: сотрудник звонит или пишет ответственному, тот смотрит в тетрадь или вспоминает, свободен ли нужный проектор, и записывает новую бронь. Из этого вытекают типичные проблемы:

– две брони одного проектора на пересекающееся время – обнаруживается, когда оба преподавателя пришли за проектором;

– бронь оборудования, которое находится в ремонте или уже списано;

– отменённое мероприятие не освобождает время в журнале, и оборудование простаивает;

– нет истории: нельзя узнать, кто пользовался оборудованием накануне поломки.

Отсюда требование к модулю: решение о брони должно приниматься программой по данным о всех активных бронях и о состоянии оборудования, а брони не должны удаляться, чтобы сохранялась история.

1.2 Обзор аналогов

Сравнение возможных способов приведено в таблице 1.

Таблица 1 – Сравнение способов бронирования оборудования

Критерий	**Бумажный журнал**	**Общий календарь (Outlook, «Яндекс Календарь»)**	**Разрабатываемый модуль**
Запрет пересекающихся броней	нет	нет (только предупреждение у отдельных календарей)	да, проверка до сохранения
Учёт состояния оборудования	нет	нет	да: доступно, на обслуживании, списано
Поиск свободного оборудования на период	вручную	по каждому календарю отдельно	одна страница со списком свободного
История и причины отмен	частично	нет	журнал операций по каждой брони
Требуется	тетрадь	учётные записи у всех	браузер

1.3 Требования к модулю

Функции определены заданием: «Реализовать каталог оборудования, выбор периода использования, создание и отмену бронирования и проверку доступности оборудования. Предусмотреть просмотр текущих и завершённых бронирований» (таблица 2).

Таблица 2 – Функциональные требования

Код	**Функция**	**Содержание**
Т1	Каталог оборудования	поиск по названию, инвентарному номеру и категории; добавление, изменение; смена состояния вместо удаления
Т2	Выбор периода	начало и окончание с точностью до минуты; срок не больше 14 дней
Т3	Проверка доступности	страница «Свободно ли оборудование с … по …» со списком свободных и занятых единиц и причиной занятости
Т4	Создание брони	оборудование, сотрудник, период, цель; запрет пересечения с активными бронями
Т5	Отмена брони	только активной и ещё не закончившейся, с обязательной причиной
Т6	Текущие и завершённые	вкладки «Текущие» и «Завершённые и отменённые», фильтры по оборудованию, сотруднику, дате и статусу, автоматическое завершение
Т7	Справочник сотрудников	добавление и изменение

Главное правило: нельзя забронировать оборудование на период, который пересекается с другим активным бронированием этого же оборудования.

Ограничения: окончание позже начала; начало не в прошлом; срок не больше 14 дней; нельзя бронировать оборудование «На обслуживании» и «Списано»; отменить можно только активную бронь, которая ещё не закончилась; цель обязательна; брони не удаляются, оборудование не удаляется, а списывается; вывести оборудование на обслуживание или списать нельзя, пока у него есть текущие брони.

Нефункциональные требования: .NET 10, ASP.NET Core Razor Pages; SQLite; русский интерфейс и сообщения; неверные данные и неизвестные записи не приводят к сбою; логика отделена от страниц и покрыта тестами; время в логике берётся из заменяемого источника, чтобы правила с «прошлым» и «завершением» можно было проверять тестами.

1.4 Выбор средств разработки

Средства разработки приведены в таблице 3.

Таблица 3 – Средства разработки

Средство	Версия	Обоснование
C#, .NET SDK	14, 10.0	платформа с долгосрочной поддержкой, изучалась на занятиях
ASP.NET Core Razor Pages	10.0	каждая страница – пара «разметка + модель страницы»; для приложения, состоящего из форм и списков, проще, чем MVC [10]
Entity Framework Core, SQLite	10.0.15	хранение в одном файле, миграции, LINQ-запросы [11]
TimeProvider	входит в .NET	единый источник текущего времени, заменяемый в тестах [12]
Bootstrap	5.3.3	оформление страниц без собственного CSS-фреймворка [13]
xUnit	2.9.3	автоматические тесты
Git, GitHub Actions	–	история версий, сборка, тесты и ручные проверки на сервере

Razor Pages выбраны вместо MVC осознанно: в модуле нет общих действий, которые удобно группировать в контроллеры, – каждая страница (каталог, бронь, доступность, график) самостоятельна, и её код лежит рядом с разметкой. Для проверки HTTP-запросами добавлено несколько методов минимального API (Minimal API) в адресах /api/....

Выводы по разделу 1

Проанализирован процесс использования общего оборудования и выявлены проблемы: пересекающиеся брони, бронь неисправного оборудования, отсутствие истории. Сравнение с журналом и календарями показало, что нужен модуль, который учитывает состояние оборудования и запрещает пересечения. Сформулированы семь функциональных требований, главное правило и ограничения, выбраны Razor Pages на .NET 10, EF Core с SQLite и TimeProvider.

2 ПРОЕКТИРОВАНИЕ

2.1 Модель данных

Модель содержит четыре сущности (рисунок 1): оборудование (Equipment), сотрудник (Employee), бронь (Booking) и запись журнала (BookingHistory).

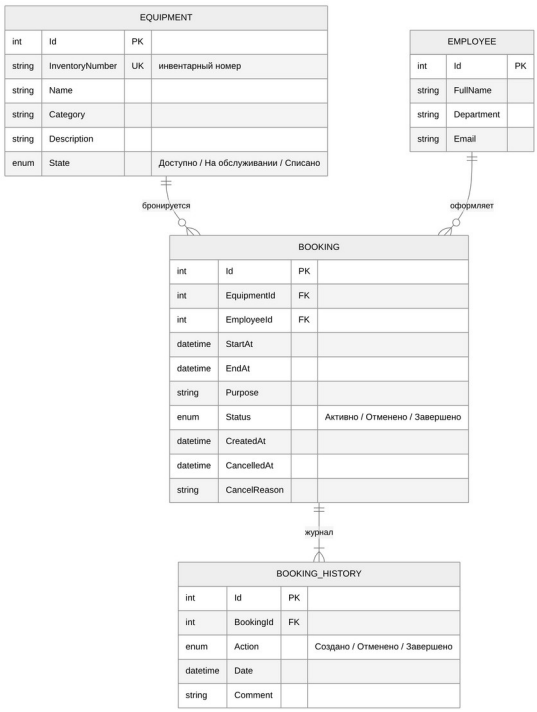


Рисунок 1 – ER-диаграмма модели данных

Описание сущностей приведено в таблице 4.

Таблица 4 – Сущности модели данных

Сущность	Поля	Ограничения
Equipment	Id, InventoryNumber, Name, Category, Description, State	инвентарный номер уникален; состояние: доступно, на обслуживании, списано
Employee	Id, FullName, Department, Email	ФИО и подразделение обязательны
Booking	Id, EquipmentId, EmployeeId, StartAt, EndAt, Purpose, Status, CreatedAt, CancelledAt, CancelReason	EndAt > StartAt; не удаляется; статус: активно, отменено, завершено
BookingHistory	Id, BookingId, Action, Date, Comment	создаётся при каждом изменении статуса брони

Связи: оборудование 1 – N броней, сотрудник 1 – N броней, бронь 1 – N записей журнала. Для связей броней с оборудованием и сотрудниками задано правило Restrict: база не позволит удалить запись, на которую ссылаются брони. Для быстрой проверки пересечений создан составной индекс (EquipmentId, StartAt, EndAt).

2.2 Главное правило и граничные случаи

Новая бронь с периодом [Sн; Eн) пересекается с существующей бронью [Sс; Eс), если выполняется условие (1):

$$S_н < E_с \ \wedge \ E_н > S_с \qquad (1)$$

где Sн, Eн – начало и окончание новой брони;

Sс, Eс – начало и окончание существующей активной брони того же оборудования.

Периоды рассматриваются как полуинтервалы: момент окончания в бронь не входит. Поэтому бронь 12:00–14:00 после брони 10:00–12:00 допустима – условие Sн < Eс (12:00 < 12:00) ложно. Пересечение хотя бы на одну минуту (11:59–14:00) делает условие истинным, и бронь отклоняется. Отменённые и завершённые брони в проверке не участвуют.

Формула (1) реализована функцией BookingRules.Overlaps (листинг 1), а проверка самого периода – функцией ValidatePeriod. Обе функции «чистые»: не обращаются к базе и к часам, поэтому проверяются тестами напрямую.

Листинг 1 – Правило пересечения и проверка периода (Services/BookingRules.cs)

```
public static readonly TimeSpan MaxDuration = TimeSpan.FromDays(14);

public static bool Overlaps(DateTime newStart, DateTime newEnd, DateTime
existingStart, DateTime existingEnd)
{
    return newStart < existingEnd && newEnd > existingStart;
}

public static string? ValidatePeriod(DateTime start, DateTime end, DateTime
now)
{
    if (end <= start)
        return "Окончание брони должно быть позже начала";
    if (start < now.AddMinutes(-1))           // минута запаса на отправку
формы
        return "Нельзя бронировать на время в прошлом";
    if (end - start > MaxDuration)
        return $"Бронь не может быть длиннее {MaxDuration.TotalDays:0} дней";
    return null;                                  // период допустим
}
```

Примеры работы формулы при существующей брони 11:00–13:00 приведены в таблице 5.

Таблица 5 – Примеры проверки пересечения с бронью 11:00–13:00

Новый период	**Sн < Ec**	**Eн > Sc**	**Результат**
10:00–12:00	10:00 < 13:00 – да	12:00 > 11:00 – да	пересекается, отказ
13:00–15:00	13:00 < 13:00 – нет	15:00 > 11:00 – да	не пересекается, бронь допустима
09:00–11:00	09:00 < 13:00 – да	11:00 > 11:00 – нет	не пересекается, бронь допустима
12:59–15:00	12:59 < 13:00 – да	15:00 > 11:00 – да	пересечение на 1 минуту, отказ
11:30–12:30	11:30 < 13:00 – да	12:30 > 11:00 – да	вложена в бронь, отказ

Срок брони ограничен условием (2):

$$E_н - S_н \le 14 \text{ сут.} \qquad (2)$$

где $E_н - S_н$ – длительность новой брони.

2.3 Состояния брони

Бронь создаётся в статусе «Активно». Из него возможны два перехода (рисунок 2): отмена пользователем с указанием причины – в статус «Отменено» и

автоматическое завершение, когда время окончания прошло, – в статус «Завершено». Обратных переходов нет.

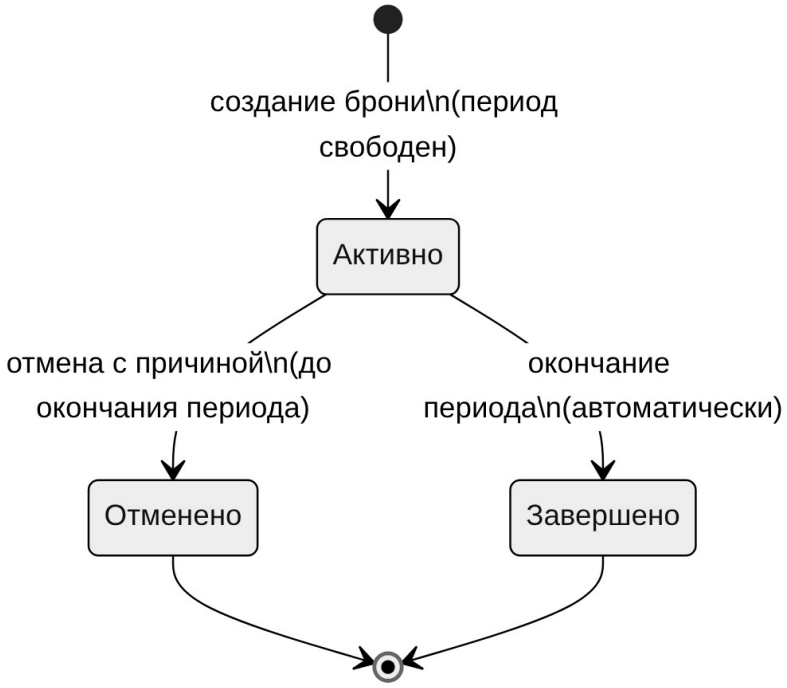


Рисунок 2 – Диаграмма состояний брони

Автоматическое завершение выполняет метод BookingService.CompleteExpired (листинг 2), который вызывается перед построением любого списка броней. Отдельная фоновая служба не нужна: статус обновляется в тот момент, когда пользователь на него смотрит.

Листинг 2 – Автоматическое завершение броней (Services/BookingService.cs)

```
public int CompleteExpired()
{
    var now = Now;                                      // время из
TimeProvider
    var expired = _db.Bookings
        .Where(b => b.Status == BookingStatus.Active && b.EndAt <= now)
        .ToList();
    foreach (var b in expired)
    {
        b.Status = BookingStatus.Completed;
        _db.BookingHistory.Add(new BookingHistory
        {
            BookingId = b.Id, Action = BookingAction.Completed, Date = now,
            Comment = "Завершено автоматически по окончании периода"
        });
    }
    if (expired.Count > 0) _db.SaveChanges();
    return expired.Count;
}
```

Пример: бронь на 13.11.2026 с 10:00 до 12:00. В 11:00 она остаётся активной (12:00 > 11:00), а при первом открытии списка после 12:00 получает статус «Завершено» и запись в журнале.

2.4 Архитектура

Архитектура модуля показана на рисунке 3. Страницы Razor Pages и методы API принимают запрос и вызывают сервисы; правила находятся в BookingService и в статическом классе BookingRules; доступ к данным – через AppDbContext.

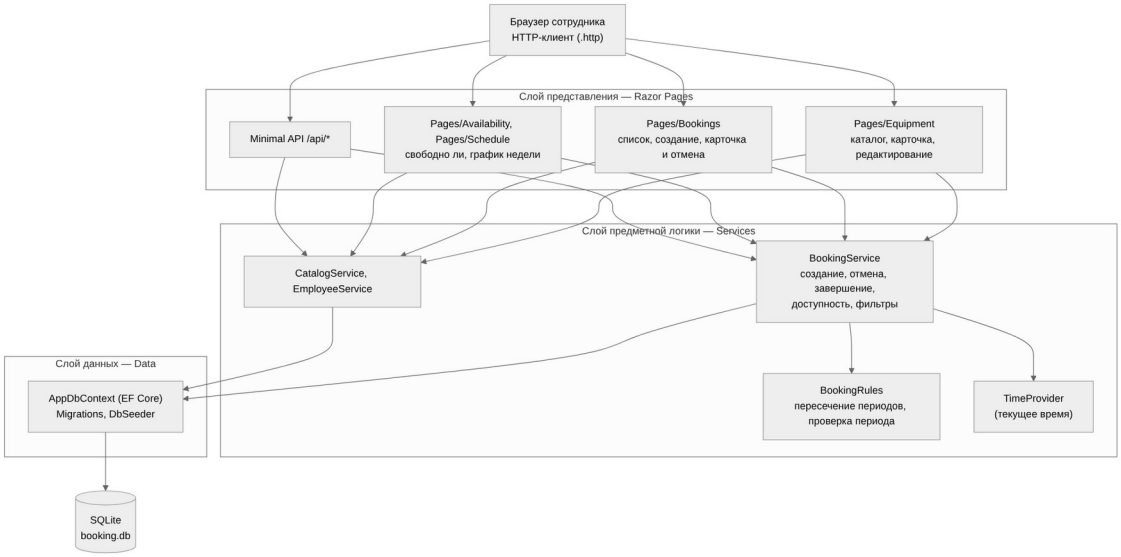


Рисунок 3 – Архитектура модуля

Особенность архитектуры – источник времени TimeProvider. Сервисы не обращаются к DateTime.Now напрямую, а получают текущее время у TimeProvider, зарегистрированного в контейнере зависимостей. В приложении это системные часы, а в тестах – «остановленные» часы FixedClock, которые тест сдвигает вперёд, чтобы проверить автоматическое завершение и запрет брони «в прошлом».

Сервисы возвращают объект OperationResult с видом ошибки: Invalid (неверные данные), NotFound (нет записи), Conflict (нарушено правило). Страница показывает текст ошибки над формой, а API превращает вид ошибки в код ответа 400, 404 или 409.

2.5 Алгоритм создания брони

Порядок проверок при создании брони показан на рисунке 4. Сначала выполняются проверки, не требующие базы (цель, период), затем проверяется оборудование и только потом ищется пересечение.

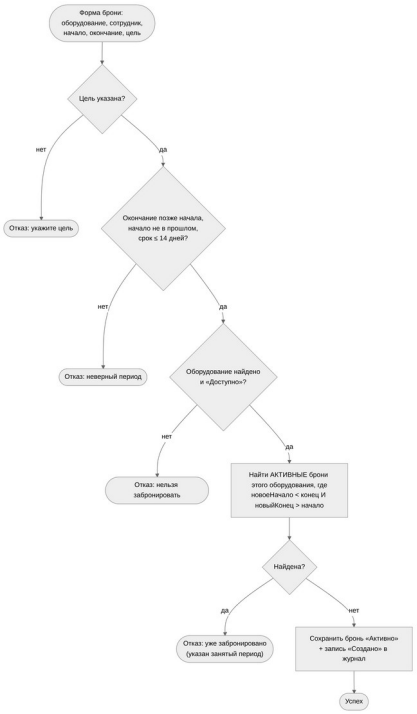


Рисунок 4 – Блок-схема алгоритма создания брони

Поиск пересечения и сохранение брони выполняются внутри критической секции lock. Если два сотрудника одновременно бронируют один проектор на одно время, второй запрос выполнит проверку после сохранения первого и получит отказ.

Выводы по разделу 2

Спроектирована модель из четырёх сущностей с журналом операций по каждой брони. Главное правило записано формулой (1) с явным разбором граничных случаев: касание концами допустимо, пересечение на минуту – нет. Определены три состояния брони и переходы между ними, в том числе автоматическое завершение. Выбрана трёхслойная архитектура с заменяемым источником времени, разработан алгоритм создания брони с защитой от одновременных запросов.

3 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ

3.1 Структура проекта и каталог оборудования

Назначение папок проекта приведено в таблице 6.

Таблица 6 – Структура проекта EquipmentBooking

Папка	Содержимое
Models	Equipment, Employee, Booking, BookingHistory, перечисления EquipmentState, BookingStatus, BookingAction
Data	AppDbContext, DbSeeder (учебные данные от даты первого запуска), Migrations
Services	BookingService, BookingRules, CatalogService, EmployeeService, OperationResult
Forms	BookingForm, BookingFilter, AvailabilityRow, WeekRow
Pages	Equipment, Bookings, Employees, Availability (доступность), Schedule (график), NotFound
EquipmentBooking.Tests	тесты xUnit: правила, сервис броней, поиск и каталог

Каталог (рисунок 5) выводится карточками с состоянием оборудования. Поиск выполняется по названию, инвентарному номеру и категории без учёта регистра, дополнительно доступны фильтры по категории и состоянию. Оборудование не удаляется: его можно вывести на обслуживание или списать, но только если у него нет текущих броней. Списанное оборудование в работу не возвращается.

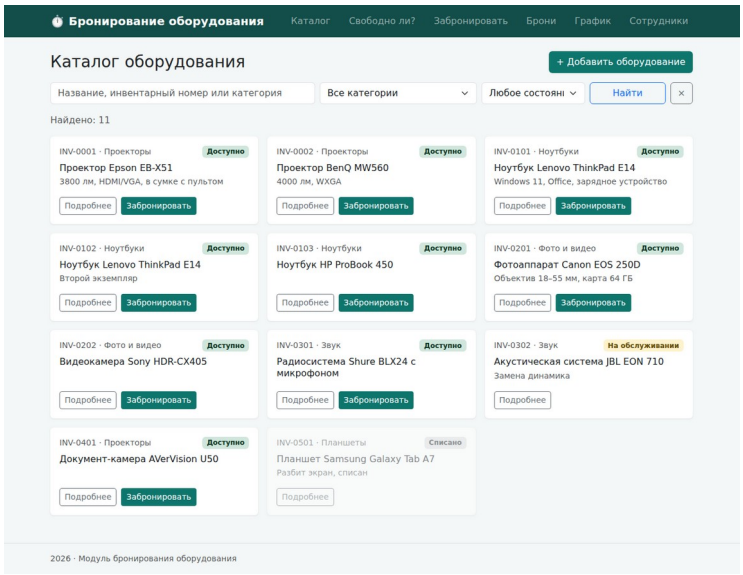


Рисунок 5 – Каталог оборудования

3.2 Проверка доступности и создание брони

Страница «Свободно ли?» (рисунок 6) принимает начало и окончание периода и делит каталог на свободное и недоступное оборудование. Для недоступного указана причина: состояние или конкретная пересекающаяся бронь с фамилией сотрудника. У каждой свободной единицы есть кнопка «Забронировать на этот период», которая открывает форму с уже заполненным периодом.

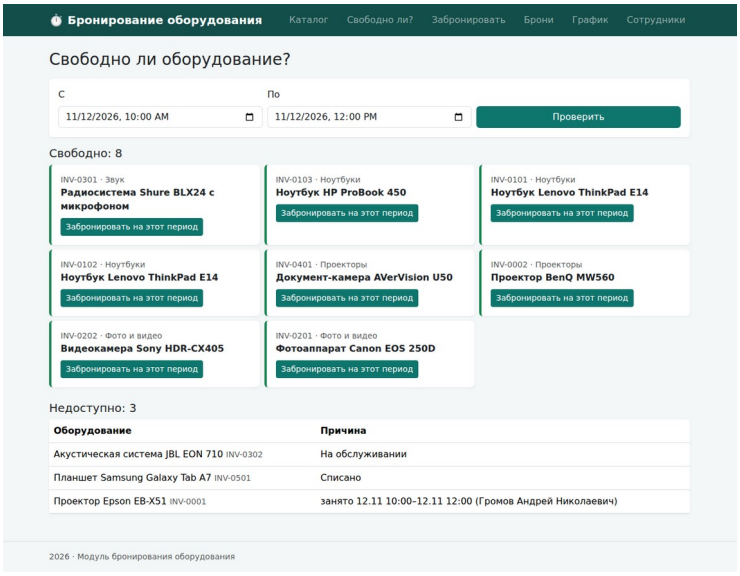


Рисунок 6 – Проверка доступности оборудования на период

Главное правило реализовано в методе BookingService.Create (листинг 3).

Листинг 3 – Поиск пересекающейся активной брони (Services/BookingService.cs, метод Create)

```
var conflict = _db.Bookings
    .Include(b => b.Employee)
    .Where(b => b.EquipmentId == form.EquipmentId && b.Status ==
BookingStatus.Active)
    .Where(b => form.StartAt < b.EndAt && form.EndAt > b.StartAt)
    .OrderBy(b => b.StartAt)
    .FirstOrDefault();
if (conflict != null)
    return OperationResult.Conflict($"«{equipment.Name}» уже забронировано с
{conflict.StartAt:dd.MM.yyyy HH:mm} ...");
```

Условие во втором Where – это формула (1). Сообщение об отказе называет занятый период и сотрудника, чтобы пользователь мог договориться или выбрать другое время (рисунок 7).

Пример из проверки: проектор INV-0001 забронирован на завтра с 10:00 до 12:00. При попытке забронировать его с 11:00 до 13:00 модуль ответил: ««Проектор Epson EB-X51» уже забронировано с 12.11.2026 10:00 по 12.11.2026 12:00 (Громов Андрей Николаевич). Выберите другое время или другое оборудование».

Модель страницы создания брони (листинг 4) только передаёт форму сервису и показывает результат: при успехе – переход на карточку новой брони, при ошибке – та же форма с сообщением.

Листинг 4 – Модель страницы создания брони (Pages/Bookings/Create.cshtml.cs, метод OnPost)

```
public IActionResult OnPost()
{
    if (ModelState.IsValid)                          // проверки атрибутов
формы
    {
        var result = _bookings.Create(Form);         // все правила — в
сервисе
        if (result.Success)
        {
            TempData["Message"] = "Бронь создана";
            return RedirectToPage("Details", new { id = result.CreatedId });
        }
        ModelState.AddModelError(string.Empty, result.Error!);
    }
    LoadLists();                                     // списки оборудования и
сотрудников
    return Page();                                   // форма с введёнными
данными и ошибкой
}
```

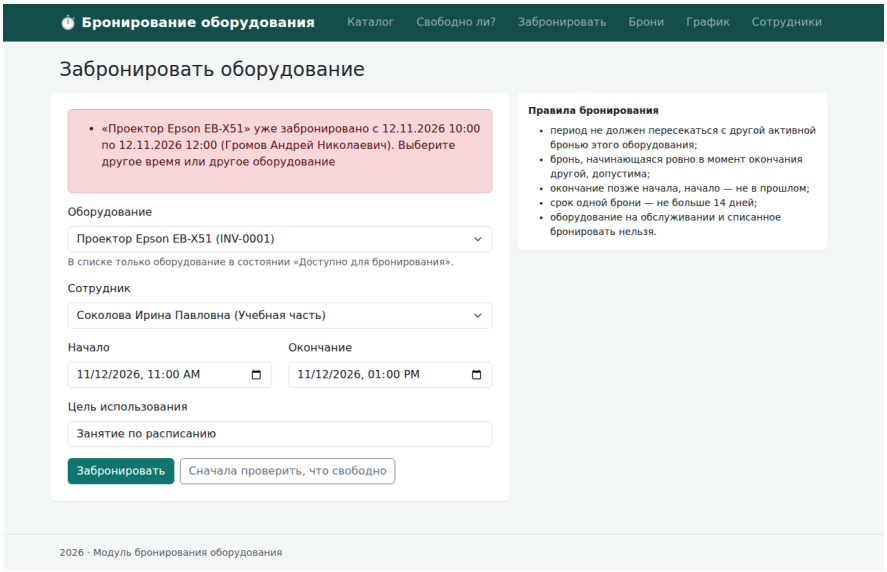


Рисунок 7 – Отклонение брони на занятый период

3.3 Отмена, текущие и завершённые брони

На карточке активной брони есть форма отмены с обязательной причиной. Метод отмены приведён в листинге 5.

Листинг 5 – Отмена брони с причиной (Services/BookingService.cs, метод Cancel)

```
public OperationResult Cancel(int bookingId, string? reason)
{
    var now = Now;
    var booking = _db.Bookings.FirstOrDefault(b => b.Id == bookingId);
    if (booking == null)
        return OperationResult.NotFound("Бронь не найдена");

    var error = BookingRules.CanCancel(booking, now);     // только активная
и не закончившаяся
    if (error != null)
        return OperationResult.Conflict(error);

    var text = (reason ?? "").Trim();
    if (text.Length < 3)
        return OperationResult.Invalid("Укажите причину отмены (не короче 3
символов)");

    booking.Status = BookingStatus.Cancelled;
    booking.CancelledAt = now;
    booking.CancelReason = text;
    _db.BookingHistory.Add(new BookingHistory
        { BookingId = booking.Id, Action = BookingAction.Cancelled, Date =
now, Comment = text });
    _db.SaveChanges();
    return OperationResult.Ok();
}
```

Пример: при повторной отмене той же брони через API сервер ответил «HTTP 409: Бронь уже отменена». После отмены статус становится «Отменено», сохраняются дата и причина, в журнал добавляется запись (рисунок 8). Отменённая бронь больше не мешает: на её время можно создать новую.

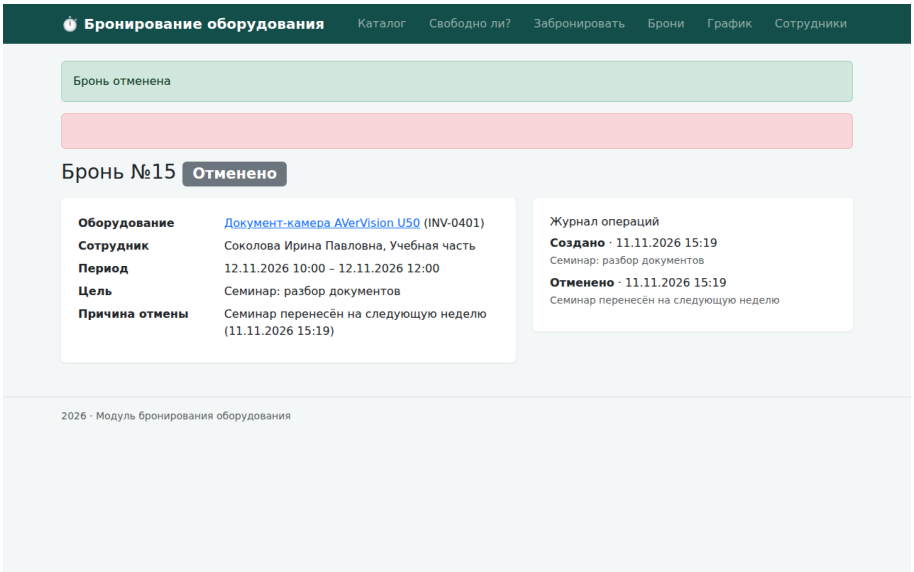


Рисунок 8 – Карточка отменённой брони с журналом операций

Список броней разделён на вкладки «Текущие» (активные, окончание ещё не наступило) и «Завершённые и отменённые». Фильтры по оборудованию, сотруднику, дате и статусу собираются в один запрос LINQ; фильтр по дате отбирает брони, которые хотя бы частично приходятся на выбранный день. Недельный график загрузки (рисунок 9) показывает занятые отрезки времени по каждой единице оборудования.

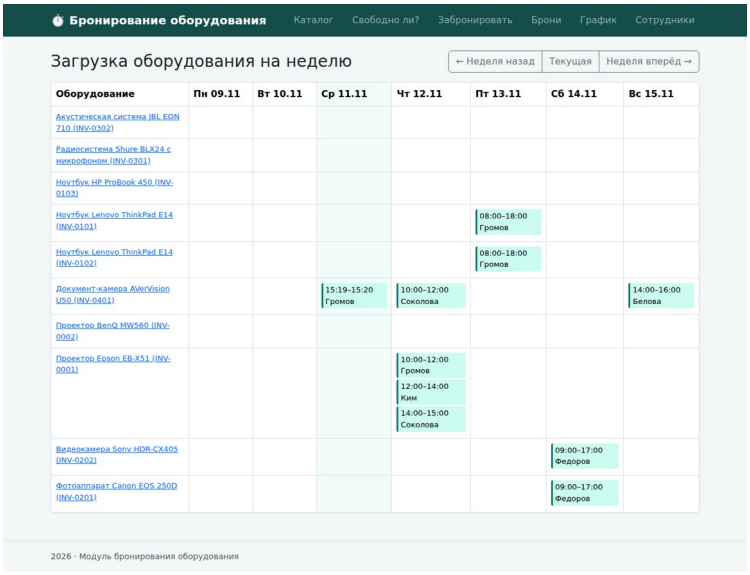


Рисунок 9 – Недельный график загрузки оборудования

3.4 Хранение данных и развёртывание

База SQLite создаётся миграцией InitialCreate при запуске (Database.Migrate()). При пустой базе DbSeeder добавляет 11 единиц

оборудования (одна на обслуживании, одна списана), 8 сотрудников и 14 броней; даты броней отсчитываются от дня первого запуска, поэтому в учебных данных всегда есть и прошедшие, и предстоящие брони. Для возврата учебных данных достаточно удалить файл booking.db. Для развёртывания нужен только .NET SDK 10 (порядок запуска – в файле README.md).

3.5 Тестирование программного модуля

Подход. Автотесты проверяют логику без браузера: «чистые» функции BookingRules и сервисы на базе SQLite в памяти. Для каждого теста создаётся новая база и «остановленные» часы FixedClock (12.11.2026 09:00), поэтому результат не зависит от момента запуска. Интерфейс, перезапуск и развёртывание проверены вручную в браузере.

Тест граничного случая «касание концами» приведён в листинге 6: после брони 10:00–12:00 бронь 12:00–14:00 должна быть создана.

Листинг 6 – Тест «бронь начинается в момент окончания другой» (EquipmentBooking.Tests/BookingServiceTests.cs)

```
[Fact]
public void СоздатьБронь_НачинаетсяВМоментОкончанияДругой_Допустимо()
{
    Book(10, 12);                // подготовка: проектор занят завтра 10:00–
12:00

    var result = Book(12, 14);   // действие: бронь 12:00–14:00

    Assert.True(result.Success); // проверка: бронь создана
}
```

Состав тестов приведён в таблице 7, полный код – в проекте EquipmentBooking.Tests, тесты правил – в приложении Б.

Таблица 7 – Состав автоматических тестов

Класс	Что проверяется	Проверок	Результат
BookingRulesTests	формула пересечения на семи вариантах (касание с обеих сторон, 1 минута, вложение, накрытие); проверка периода, включая ровно 14 дней и 14 дней + 1 минута; возможность отмены в четырёх состояниях	16	пройдены
BookingServiceTests	создание; пересечение; касание; 1 минута; отменённая бронь не мешает; другое оборудование; обслуживание и списание; пустая цель; неверный период; неизвестные Id; отмена, повторная отмена, без причины, после окончания; автоматическое завершение до и после окончания	21	пройдены
SearchAndCatalogTests	текущие и завершённые; фильтры; доступность на период и на границе; поиск в каталоге; повтор инвентарного номера; списание при текущих бронях; возврат списанного; неизвестный Id; чтение новым контекстом	15	пройдены

Тесты запущены на сервере GitHub Actions (Ubuntu, .NET 10): всего 52, пройдено 52, не пройдено 0 (рисунок 10).

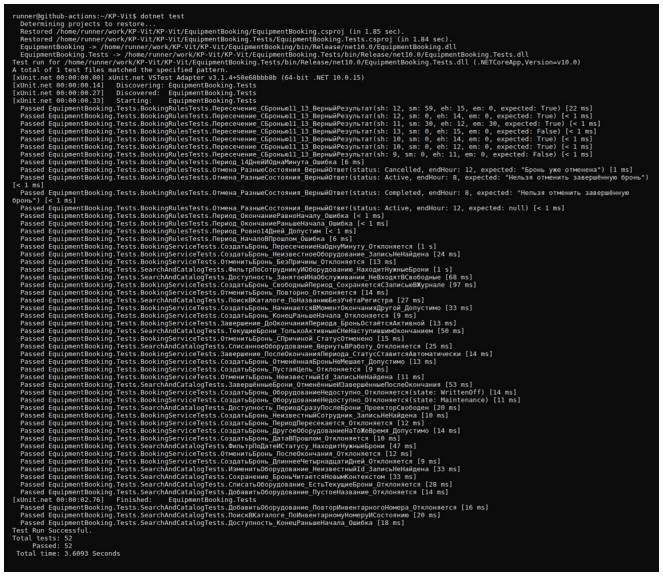


Рисунок 10 – Результаты выполнения автоматических тестов

Ручные проверки выполнены автоматизированным браузером по сценарию tools/screenshots.py на копии проекта, развёрнутой из архива: сценарий заполняет формы так же, как пользователь, и записывает сообщение, которое реально появилось на странице. Всего 31 проверка, основные приведены в таблице 8, полная таблица оформлена отдельным документом «Таблица проверок».

Таблица 8 – Основные ручные проверки

№	Проверка	Ожидаемый результат	Фактический результат
8	создать бронь на свободный период	бронь создана	Бронь создана → Бронь №15 Активно
9	проектор 11:00–13:00 при брони 10:00–12:00	отказ	«Проектор Epson EB-X51» уже забронировано с 12.11.2026 10:00 по 12.11.2026 12:00 (Громов Андрей Николаевич). Выберите другое время или другое оборудование
11	14:00–15:00 сразу после 12:00–14:00	бронь создана	Бронь создана
19	отменить с причиной	статус «Отменено»	Бронь отменена → статус: Отменено
21	новая бронь на время отменённой	бронь создана	Бронь создана
22	бронь на 1 минуту, ожидание 70 с	статус «Завершено»	создана 201; через 70 с статус: Завершено; журнал: Создано / Завершено
29	обслуживание при текущих бронях	отказ	У оборудования есть текущие брони (3). Сначала отмените их
31	перезапуск приложения	данные на месте	INV-0601 найден: да; текущие брони: Найдено: 10 (до перезапуска: Найдено: 10)

Найденные при разработке ошибки и исправления приведены в таблице 9.

Таблица 9 – Найденные ошибки и исправления

№	Ошибка	Причина	Исправление	Повторная проверка
1	ошибка при отмене уже отменённой брони не показывалась на странице	сообщение выводилось внутри формы отмены, а форма у отменённой брони скрыта	сообщение перенесено над карточкой брони	проверка 20 (через API) и отмена на странице показывают текст ошибки
2	в файле решения у основного проекта был идентификатор, совпадающий с типом проекта	ошибка при ручном составлении .sln	присвоен уникальный идентификатор	решение собирается, тесты проходят
3	при запуске без миграции таблицы не создавались	Database.Migrate() без файлов миграции ничего не делает	создана миграция InitialCreate	база создаётся при чистом запуске (проверка на копии из архива)

Оставшиеся ограничения: нет входа в систему и ролей (заданием не предусмотрены); автоматическое завершение выполняется при открытии

страниц, а не по расписанию; защита от одновременных броней действует в пределах одного экземпляра приложения; стили Bootstrap загружаются из сети.

Выводы по тестированию: все 52 автоматические и 31 ручная проверка дали ожидаемый результат; главное правило и все граничные случаи из задания подтверждены тестами и проверками в браузере.

Выводы по разделу 3

Реализованы все функции задания: каталог с поиском и сменой состояния, проверка доступности на период, создание брони с проверкой пересечений, отмена с причиной, текущие и завершённые брони с фильтрами, автоматическое завершение, недельный график. Логика проверена 52 автотестами, интерфейс – 31 ручной проверкой на копии из архива.

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработан модуль бронирования оборудования. Цель достигнута; это подтверждается решением каждой задачи.

Процесс бронирования изучен в разделе 1: выявлены проблемы пересекающихся броней и брони неисправного оборудования, рассмотрены журнал и общие календари, сформулированы семь функций, главное правило и ограничения.

Проектирование выполнено в разделе 2: модель из четырёх сущностей с журналом операций, формула пересечения с разбором граничных случаев, три состояния брони, трёхслойная архитектура с заменяемым источником времени, алгоритм создания брони.

Реализация описана в подразделах 3.1–3.4: модуль на Razor Pages с каталогом, страницей доступности, созданием и отменой броней, вкладками текущих и завершённых броней, фильтрами и графиком загрузки.

Проверка выполнена в подразделе 3.5: 52 автоматических и 31 ручная проверка пройдены, найденные ошибки исправлены.

Инструкции подготовлены: README.md проекта и материалы к защите. Код правил бронирования приведён в приложениях А и Б.

Перспективы развития: вход в систему и роли «сотрудник» и «ответственный»; напоминания по электронной почте о начале брони; учёт выдачи и возврата оборудования с отметкой о состоянии; повторяющиеся брони (каждый вторник); фоновая служба автоматического завершения.

«___» _____ 2026 г. _____ / _____

(подпись) (ФИО)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : издание официальное : дата введения 1992-01-01. – Москва : Стандартинформ, 2010. – 22 с.

2. Об утверждении Федерального стандарта бухгалтерского учета ФСБУ 6/2020 «Основные средства» : Приказ Министерства финансов Российской Федерации от 17 сентября 2020 г. № 204н. – Текст : электронный // Официальный интернет-портал правовой информации. – URL: http://pravo.gov.ru (дата обращения: 11.11.2026).

3. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : межгосударственный стандарт : издание официальное : дата введения 2022-01-01. – Москва : Стандартинформ, 2021. – 15 с.

4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с.

5. Прайс, М. C# 10 и .NET 6. Современная кросс-платформенная разработка / М. Прайс. – Санкт-Петербург : Питер, 2023. – 848 с. [проверить]

6. Смит, Д. Entity Framework Core в действии / Д. Смит ; перевод с английского. – Москва : ДМК Пресс, 2022. – 690 с. [проверить]

7. Хориков, В. Принципы юнит-тестирования / В. Хориков. – Санкт-Петербург : Питер, 2021. – 320 с.

8. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с.

9. Lock, A. ASP.NET Core in Action / A. Lock. – 3rd ed. – Shelter Island : Manning, 2023. – 984 p. [проверить]

10. Razor Pages architecture and concepts in ASP.NET Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/razor-pages/ (дата обращения: 11.11.2026).

11. Entity Framework Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ef/core/ (дата обращения: 11.11.2026).

12. What is TimeProvider? – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/dotnet/standard/datetime/timeprovider-overview (дата обращения: 11.11.2026).

13. Bootstrap. Documentation. – Текст : электронный // Bootstrap. – URL: https://getbootstrap.com/docs/5.3/ (дата обращения: 11.11.2026).

Приложение А

Листинг правил бронирования

Файл Services/BookingRules.cs

```
using EquipmentBooking.Models;

namespace EquipmentBooking.Services;

public static class BookingRules
{
    public static readonly TimeSpan MaxDuration = TimeSpan.FromDays(14);

    public static bool Overlaps(DateTime newStart, DateTime newEnd, DateTime
existingStart, DateTime existingEnd)
    {
        return newStart < existingEnd && newEnd > existingStart;
    }

    public static string? ValidatePeriod(DateTime start, DateTime end, DateTime
now)
    {
        if (end <= start)
            return "Окончание брони должно быть позже начала";
        // Минута запаса: форма отправляется не мгновенно
        if (start < now.AddMinutes(-1))
            return "Нельзя бронировать на время в прошлом";
        if (end - start > MaxDuration)
            return $"Бронь не может быть длиннее {MaxDuration.TotalDays:0}
дней";
        return null;
    }

    public static bool IsCurrent(Booking b, DateTime now) => b.Status ==
BookingStatus.Active && b.EndAt > now;

    public static string? CanCancel(Booking b, DateTime now)
    {
        if (b.Status == BookingStatus.Cancelled)
            return "Бронь уже отменена";
        if (b.Status == BookingStatus.Completed || b.EndAt <= now)
            return "Нельзя отменить завершённую бронь";
        return null;
    }
}
```

Приложение Б

Листинг тестов правил бронирования

Файл EquipmentBooking.Tests/BookingRulesTests.cs

```
using EquipmentBooking.Models;
using EquipmentBooking.Services;

namespace EquipmentBooking.Tests;

public class BookingRulesTests
{
    private static readonly DateTime D = TestEnv.Today;
    private static DateTime T(int h, int m = 0) => D.AddHours(h).AddMinutes(m);

    [Theory]
    [InlineData(10, 0, 12, 0, true)]   // 10:00–12:00 при брони 11:00–13:00 — 
пересекается
    [InlineData(12, 0, 14, 0, true)]   // захватывает конец существующей
    [InlineData(13, 0, 15, 0, false)]  // начинается ровно в момент окончания — 
допустимо
    [InlineData(9, 0, 11, 0, false)]   // заканчивается ровно в момент начала — 
допустимо
    [InlineData(12, 59, 15, 0, true)]  // пересечение на 1 минуту (12:59–13:00)
    [InlineData(11, 30, 12, 30, true)] // целиком внутри
    [InlineData(10, 0, 14, 0, true)]   // целиком накрывает
    public void Пересечение_СБронью11_13_ВерныйРезультат(int sh, int sm, int eh, 
int em, bool expected)
    {
        Assert.Equal(expected, BookingRules.Overlaps(T(sh, sm), T(eh, em), 
T(11), T(13)));
    }

    [Fact]
    public void Период_ОкончаниеРаньшеНачала_Ошибка()
    {
        Assert.Equal("Окончание брони должно быть позже начала", 
BookingRules.ValidatePeriod(T(12), T(10), T(9)));
    }

    [Fact]
    public void Период_ОкончаниеРавноНачалу_Ошибка()
    {
        Assert.NotNull(BookingRules.ValidatePeriod(T(12), T(12), T(9)));
    }

    [Fact]
    public void Период_НачалоВПрошлом_Ошибка()
    {
        Assert.Equal("Нельзя бронировать на время в прошлом", 
BookingRules.ValidatePeriod(T(8), T(10), T(9)));
    }

    [Fact]
    public void Период_Ровно14Дней_Допустим()
    {
        Assert.Null(BookingRules.ValidatePeriod(T(10), T(10).AddDays(14), 
T(9)));
    }

    [Fact]
```

```
    public void Период_14ДнейИОднаМинута_Ошибка()
    {
        Assert.NotNull(BookingRules.ValidatePeriod(T(10),
T(10).AddDays(14).AddMinutes(1), T(9)));
    }

    [Theory]
    [InlineData(BookingStatus.Active, 12, null)]
    [InlineData(BookingStatus.Cancelled, 12, "Бронь уже отменена")]
    [InlineData(BookingStatus.Completed, 8, "Нельзя отменить завершённую
бронь")]
    [InlineData(BookingStatus.Active, 8, "Нельзя отменить завершённую бронь")]
// окончание уже прошло
    public void Отмена_РазныеСостояния_ВерныйОтвет(BookingStatus status, int
endHour, string? expected)
    {
        var b = new Booking { Status = status, StartAt = T(7), EndAt =
T(endHour) };

        Assert.Equal(expected, BookingRules.CanCancel(b, T(9)));
    }
}
```